3.e Scope Functions

let, run, with, apply, also & more

Contents

1 **What Are Scope Functions?**
Definition, purpose & the two key differences: context object reference & return value

2 **let, run & with**
Transformation functions — return the lambda result, context as it or this

3 **apply & also**
Mutation functions — return the context object, used for configuration & side effects

4 **takeIf & takeUnless**
Conditional filtering for single objects in call chains

5 **Practice Exercises**
Solved and unsolved exercises combining all scope functions

What Are Scope Functions?

The Kotlin standard library contains several functions whose sole purpose is to execute a block of code within the context of an object.

Five scope functions: let · run · with · apply · also

Two Key Differences

1 Context Object Reference

this (lambda receiver)
→ run, with, apply

it (lambda argument)
→ let, also

2 Return Value

Lambda result
→ let, run, with

Context object
→ apply, also

Scope Functions Overview

Function	Object Ref	Return Value	Is Extension?
let	it	Lambda result	Yes
run	this	Lambda result	Yes
with	this	Lambda result	No
apply	this	Context object	Yes
also	it	Context object	Yes

💡 Key insight: The choice depends on what you do next. Need the result? Use let/run/with. Need to chain on the same object? Use apply/also.

The let Function

let

Context: it | Returns: Lambda result | Extension: Yes

Executes a block of code with the object as its argument (it) and returns the result of the lambda expression.

Common Use Cases

- Null safety checks with ?.let { }
- Transforming a value into another type
- Introducing local variables with limited scope

```
val name: String? = "Kotlin"
val length = name?.let {
    println("Name is $it")
    it.length    // returns Int
} ?: 0
println("Length: $length")  // Length: 6
```

The run Function

run Context: this | Returns: Lambda result | Extension: Yes

Like let, but the context object is available as a receiver (this). You can call its methods directly without a qualifier.

Common Use Cases

- Object configuration + computing a result
- Running statements where an expression is needed
- Non-extension variant: run { } for code blocks

```
val service = MultiportService("https://example.org", 80)
val result = service.run {
    port = 8080
    query(prepareRequest() + " to port $port")
}
println(result)
```

The with Function

with

Context: this | Returns: Lambda result | Not an extension function

A non-extension function that takes the context object as an argument. Use it for calling functions on the context object without providing the lambda result.

Common Use Cases

- Grouping function calls on a non-null object
- Read as: "with this object, do the following"
- Creating a temporary scope for helper variables

```
val numbers = mutableListOf("one", "two", "three")
with(numbers) {
    val firstItem = first()
    val lastItem = last()
    println("First: $firstItem, Last: $lastItem")
}
```

The apply Function

apply Context: this | Returns: Context object | Extension: Yes

Returns the context object itself. The context is available as this. Use it for object configuration — "apply the following assignments to the object."

Common Use Cases

- Object initialization and property configuration
- Builder-like patterns for mutable objects
- Chainable: returns the object for further calls

```
val peter = Person().apply {
    name = "Peter"
    age = 25
    city = "Amsterdam"
}
println(peter)  // Person(name=Peter, age=25, city=Amsterdam)
```

The also Function

also

Context: it | Returns: Context object | Extension: Yes

Returns the context object. The context is available as it. Use it for side effects that don't alter the object — "and also do the following."

Common Use Cases

- Logging and debugging in call chains
- Performing side effects without changing the object
- Chaining with apply for config + validation

```
fun getRandomInt(): Int {
    return Random.nextInt(100).also { value ->
        writeToLog("getRandomInt() generated value $value")
    }
}
val i = getRandomInt()  // logs and returns the value
```

takeIf & takeUnless

In addition to scope functions, the standard library provides **takeIf** and **takeUnless** for embedding object state checks in call chains.

takeIf

Returns the object if the predicate is true, otherwise returns null.

Acts as a filter for a single object.

takeUnless

Opposite of takeIf: returns the object if the predicate is false, otherwise returns null.

```
val number = Random.nextInt(100)
val evenOrNull = number.takeIf { it % 2 == 0 }
val oddOrNull = number.takeUnless { it % 2 == 0 }
println("even: $evenOrNull, odd: $oddOrNull")

// Combining with scope functions
val str = "Hello"
str.takeIf { it.isNotEmpty() }?.let { println(it.uppercase()) }
```

Choosing the Right Scope Function

apply	**Object configuration**	Initialize or configure an object
also	**Side effects**	Logging, validation, debugging in chains
let	**Null safety + transform**	Execute code on a nullable object, return new value
run	**Compute on receiver**	Run operations with this and return a result
with	**Group calls on object**	Call multiple methods on a non-null object

Avoid overusing scope functions — they can make code hard to read. Avoid nesting them and be mindful of **this** vs **it** confusion.

Chaining Scope Functions

apply and **also** return the context object, so they can be chained as "side steps" in longer call chains.
let, **run**, and **with** return the lambda result and can be used to assign computed values.

Chaining apply + also

```
val numberList = mutableListOf<Double>()
numberList.also { println("Populating the list") }
    .apply { add(2.71); add(3.14); add(1.0) }
    .also { println("Sorting the list") }
    .sort()
```

Using also in return statements

```
fun getRandomInt(): Int {
    return Random.nextInt(100).also {
        writeToLog("getRandomInt() generated value $it")
    }
}
```

Use takeIf/takeUnless with let for conditional execution: obj.takeIf { condition }?.let { doSomething(it) }

Practice Exercises

Practice Exercises

Test your understanding of Kotlin scope functions with these exercises. Solved exercises include a walkthrough; unsolved exercises are for you to try on your own.

1. **Solved: Refactor with let**
   Null-safe string transformation

2. **Solved: Object setup with apply**
   Configure a data class instance

3. **Solved: Chaining also + apply**
   Build a list with logging

4. **Unsolved: Pick the right function**
   Choose let, run, with, apply, or also

5. **Unsolved: Refactor challenge**
   Rewrite verbose code using scope functions

Exercise 1: Refactor with let

SOLVED

Refactor the following code to use let for null-safe string processing:

Before (verbose):

```
val input: String? = getUserInput()
var result = "default"
if (input != null) { result = input.trim().uppercase() }
println(result)
```

After (using let):

```
val input: String? = getUserInput()
val result = input?.let { it.trim().uppercase() } ?: "default"
println(result)
```

Explanation:

- **let** executes the block only when input is non-null (?.let)
- Inside the lambda, **it** refers to the non-null input
- The Elvis operator (?:) provides the fallback for the null case
- The result is the lambda return value (the transformed string)

Exercise 2: Object Setup with apply

SOLVED

Refactor object initialization to use apply instead of repeated variable references:

Before (verbose):

```
val car = Car()
car.brand = "Toyota"; car.model = "Corolla"
car.year = 2024; car.color = "Blue"
println(car)
```

After (using apply):

```
val car = Car().apply {
    brand = "Toyota"; model = "Corolla"
    year = 2024; color = "Blue"
}
println(car)
```

Explanation:
- apply provides **this** as the receiver — no need to repeat car.
- apply returns the context object, so car is assigned the configured instance
- This is the most common use case for apply: builder-style initialization

Exercise 3: Chaining also + apply

SOLVED

Create a list with logging at each step using also and apply in a chain:

Solution:

```
val numbers = mutableListOf<Int>()
    .also { println("Empty list created: $it") }
    .apply {
        add(1); add(2); add(3)
    }
    .also { println("After adding elements: $it") }
    .apply { sort() }
    .also { println("After sorting: $it") }
```

Output:

```
Empty list created: []
After adding elements: [1, 2, 3]
After sorting: [1, 2, 3]
```

Key takeaway:
- **also** is used for logging (side effects) — it doesn't change the object
- **apply** is used for mutating the list — it gives access to this
- Both return the context object, enabling the chain to continue

Exercise 4: Pick the Right Function

UNSOLVED

For each scenario below, choose the best scope function (let, run, with, apply, or also).
Replace the ??? with the correct function name and fill in the lambda body.

A) Configure a TextView:

```
val textView = TextView(context).???{
    text = "Hello"
    textSize = 16f
    setTextColor(Color.BLACK)
}
```

Hint: We need the configured object back.

B) Null-safe transformation:

```
val email: String? = getEmail()
val domain = email?.??? {
    substringAfter("@")
}
```

Hint: We need a transformed result from a nullable.

C) Log and return a value:

```
fun createUser(): User {
    return User("Alice", 25).??? {
        println("Created user: $it")
    }
}
```

Hint: Side effect, but we need to return the object.

D) Access multiple properties:

```
val config = getConfiguration()
???(config) {
    println("Host: $host")
    println("Port: $port")
}
```

Hint: Non-null object, group calls, no return needed.

Exercise 5: Refactor Challenge

UNSOLVED

Rewrite the following verbose code using appropriate scope functions. You may use multiple scope functions, including takeIf, to make it concise and idiomatic.

Original code to refactor:

```
data class Order(var items: MutableList<String> = mutableListOf(),
                 var discount: Double = 0.0, var note: String = "")

fun processOrder(input: String?): Order? {
    if (input == null) return null
    val trimmed = input.trim()
    if (trimmed.isEmpty()) return null
    val order = Order()
    order.items.add(trimmed)
    order.discount = 0.1
    order.note = "Online order"
    println("Order created: $order")
    return order
}
```

Your refactored solution:

```
// Use let for null check, takeIf for validation,
// apply for object configuration, and also for logging.
// Try to write it as a single expression!
```